Enumerative Program Synthesis with Observational Equivalence Reduction

Varun Varma

March 25th, 2020

Background

Program synthesis is the problem of generating a program from a given specification. Enumerative program synthesis is a family of program synthesis algorithms where the search space is enumerated until a program is found. Bottom-up search is another program synthesis algorithm that generates programs from previously generated programs. Enumerative bottom-up program synthesis is a powerful program synthesis technique that produces complete programs on every iteration and allows for a variety of optimizations via heuristics. However, an issue with this technique is that it generates programs that produce the same output. For example, x and $(x * 2)/2$ are equivalent programs, yet both these programs will have to be added to the program bank for future use, leading to exponential efficiency loss. These equivalent programs can be optimized away by comparing the observed output of a program with all other generated programs and keeping only the smallest one. This technique, called observational equivalence reduction, has the potential to radically speed up enumerative bottom-up program synthesis. This project implements a bottom-up enumerative program synthesis engine with optional observational equivalence reduction and evaluates it on a benchmark of synthesis problems o test the effectiveness of observational equivalence reduction.

Materials and Methods

An enumerative program synthesis engine that could optionally apply observational equivalence reduction was benchmarked on nine bit-manipulation problems adapted from the *Hacker's Delight* book. Each problem was defined by 256 input-out pairs calculated from the desired function. The synthesizer inputs a specification of type $[(i8, i8); 256]$ and outputs the total number of programs iterated and a function of type $i8 \rightarrow i8$ that satisfies the specification. The synthesizer was evaluated on each problem in the benchmark, once with observational equivalence reduction and once without. The total synthesis time, size of synthesized program, and number of programs iterated were recorded into a .csv file. All benchmarks were run on an Intel Core i7-9750H and 32 GB of RAM on a single thread.

Data

The enumerative program synthesis engine has been rewritten completely from Python to Rust, which will provide more speed and reliable data. The nine benchmark problems have been respecified to fit the new synthesis engine. The engine supports multi-argument specifications, so more problems from the *Hacker's Delight* problem set can be evaluated if necessary. The engine also features basic heuristics for traversing the search space, which allows it to synthesize up to height-3 programs, despite the search space of height-3 programs being millions of programs large.

```
// P1 ~ Turn off rightmost 1 bit
fn problem1(x: Value) -> Value {
    x & (x - Wrapping(1))
}

// P2 ~ Test whether an unsigned integer is of form 2^(n-1)
fn problem2(x: Value) -> Value {
    x & (x + Wrapping(1))
}

// P3 ~ Isolate rightmost bit
fn problem3(x: Value) -> Value {
    x & -x
}

// P4 ~ Form a mask that identifies the rightmost 1 bit and trailing 0s
fn problem4(x: Value) -> Value {
    x ^ (x - Wrapping(1))
}

// P5 ~ Right propagate rightmost 1 bit
fn problem5(x: Value) -> Value {
    x | (x - Wrapping(1))
}

// / P6 ~ Turn on rightmost 0 bit
```

```
fn problem6(x: Value) -> Value {
    x | (x + Wrapping(1))
}

// P7 ~ Isolate rightmost 0 bit
fn problem7(x: Value) -> Value {
    (!x) & (x + Wrapping(1))
}

// P8 ~ Form a mask that identifies trailing 0s
fn problem8(x: Value) -> Value {
    (x - Wrapping(1)) & (!x)
}

// P13 ~ Sign function
fn problem13(x: Value) -> Value {
    let o1 = x.rotate_right(MAX as u32);
    let o2 = -x;
    let o3 = o2.rotate_right(MAX as u32);
    return o1 | o3;
}
```

Figure 1: The nine problems from the *Hacker's Delight* problem set.

The benchmarks have been evaluated and the data has been collected.

Problem	**Synthesis Time (µs) Without Observational Equivalence Reduction**	**Synthesis Time (µs) With Observational Equivalence Reduction**
1	47439	11745
2	11421	6040
3	592128	46845
4	1179776	83811
5	7507	4082
6	2898	2087
7	599882	48557
8	1725702	43172
9	4330286	256931

Figure 2: A table of synthesis time for nine problems with and without observational equivalence reduction.

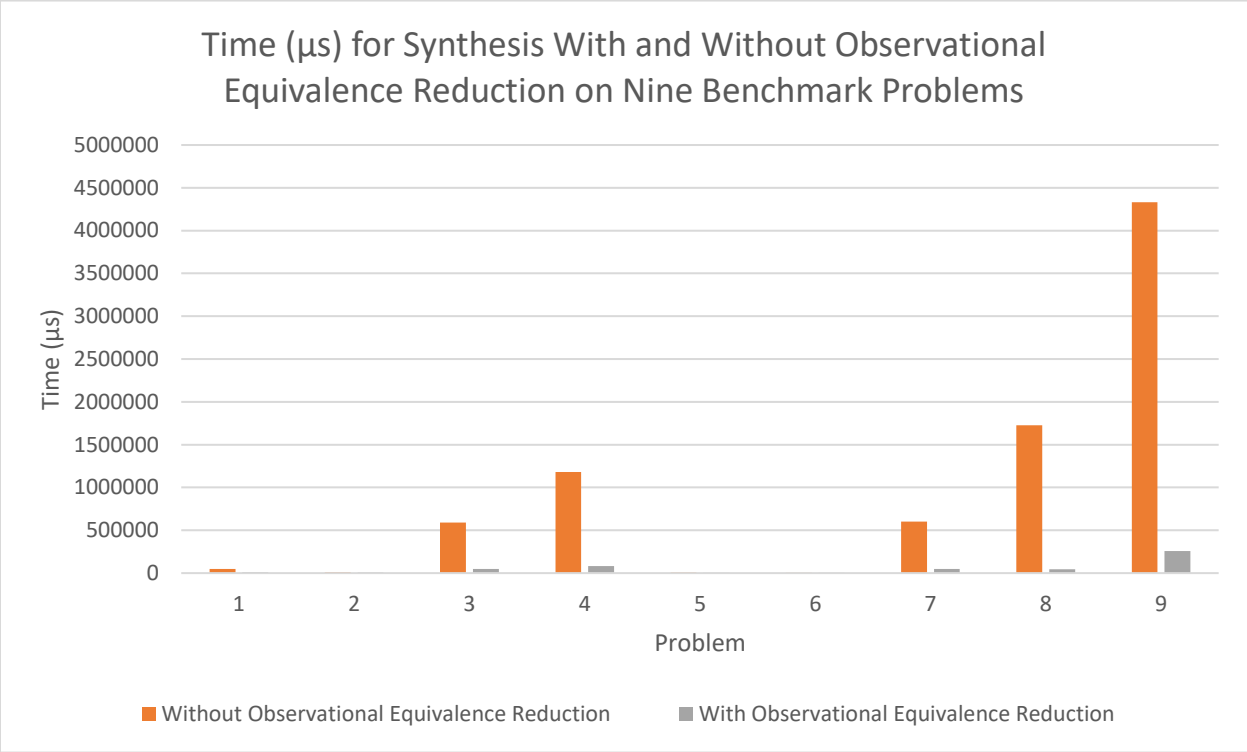

Figure 4: A graph displaying the synthesis time for nine problems with and without observational equivalence reduction.

Problem	**Program Iteration Count Without Observational Equivalence Reduction**	**Program Iteration Count With Observational Equivalence Reduction**
1	5436	876
2	1384	542
3	78965	4385
4	157700	7634
5	1097	426
6	269	255
7	72500	3925
8	187250	4385
9	497537	22795

Figure 5: A table of total program iteration count for nine problems with and without observational equivalence reduction.

The data suggests that observational equivalence reduction dramatically reduces the synthesis time of enumerative program synthesis. Expanded data analysis will involve calculating the ratio between synthesis time without and with observational equivalence reduction and looking for a link between iteration count and efficiency improvements.

Problem	**Synthesized Program Without Observational Equivalence Reduction**	**Synthesized Program With Observational Equivalence Reduction**
1	(& (- x 1) (+ x 0))	(& (- x 1) x)
2	(& (^ x 0) (+ 1 x))	(& x (+ x 1))
3	(& (/ x 0) (- 0 x))	(& x (- 0 x))
4	(^ (- 0 x) (! x))	(^ (+ x -128) (+ 127 x))
5	(| (| 1 x) (- x 1))	(| (| x 1) (- x 1))
6	(| (| 1 x) (+ 1 x))	(| (| x 1) (+ x 1))
7	(& (+ x 1) (! x))	(& (+ 1 x) (! x))
8	(& (- x 1) (! x))	(& (! x) (- x 1))
9	(& (^ (- 0 x) (^ 1 x)) (/ (- 0 x) (>> x 0)))	(& (^ (- 0 x) (^ x 1)) (/ (- 0 x) x))

Figure 6: A table of synthesized programs for nine problems with and without observational equivalence reduction.

Works Cited

Feng, Y., Martins, R., Bastani, O., & Dillig, I. (2018). Program synthesis using conflict-driven learning. *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation - PLDI 2018*. doi:10.1145/3192366.3192382

Guo, Z., James, M., Justo, D., Zhou, J., Wang, Z., Jhala, R., & Polikarpova, N. (2020). Program synthesis by type-guided abstraction refinement. *Proceedings of the ACM on Programming Languages, 4*(POPL), 1-28. doi:10.1145/3371080

Osera, P., & Zdancewic, S. (2015). Type-and-example-directed program synthesis. *ACM SIGPLAN Notices, 50*(6), 619-630. doi:10.1145/2813885.2738007

Polikarpova, N., Kuraj, I., & Solar-Lezama, A. (2016). Program synthesis from polymorphic refinement types. *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation - PLDI 2016*. doi:10.1145/2908080.2908093

Wang, X., Dillig, I., & Singh, R. (2018). Program synthesis using abstraction refinement. *Proceedings of the ACM on Programming Languages, 2*(POPL), 1-30. doi:10.1145/3158151